1. Create tensors of shapes $(8, 1)$ and $(8, 2)$; predict the result shape of matrix products before running them.

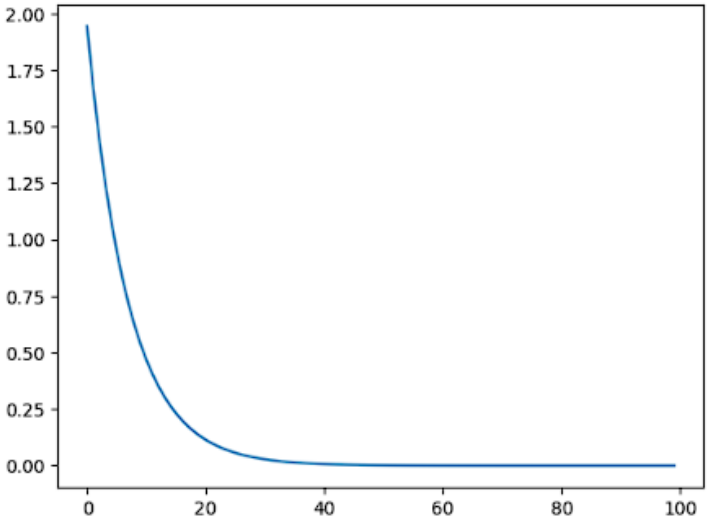
Code :

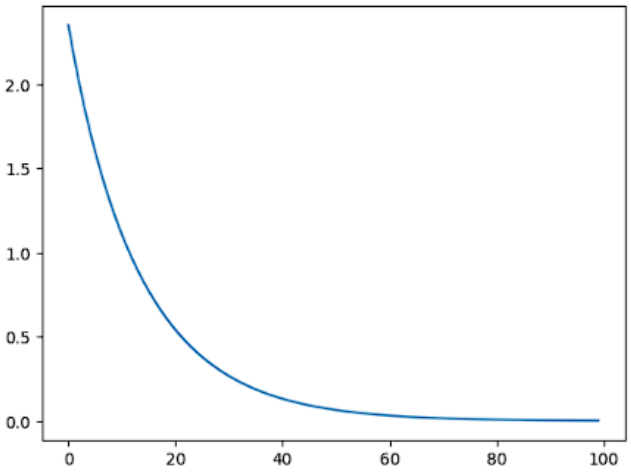
```
# practice 1
import torch
A = torch.randn(8, 1)
B = torch.randn(8, 2)
print("A shape:", A.shape)
print("B shape:", B.shape)
C = A.T @ B
print("a taranahade b shape:", C.shape)
D = B.T @ A
print("b taranahade a shape:", D.shape)

Output:
A shape: torch.Size([8, 1])
B shape: torch.Size([8, 2])
a taranahade b shape: torch.Size([1, 2])
b taranahade a shape: torch.Size([2, 1])
```

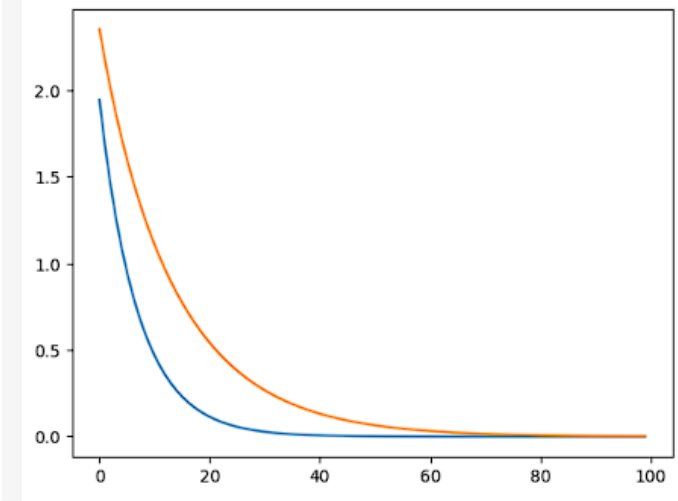
2. Fit $y = 2x - 0.4$ with one `nn.Linear`. Write the training loop yourself; plot train loss and absolute parameter error.

Code :

```
#practice 2 last try

import torch
import torch.nn as nn
import matplotlib.pyplot as plt

x = torch.linspace(-1, 1, 100).reshape(100, 1)
y = 2 * x - 0.4

model = nn.Linear(1, 1)

loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

losses = []
errors = []

for i in range(100):
    prediction = model(x)

    loss = loss_function(prediction, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    losses.append(loss.item())

    weight = model.weight.item()
    bias = model.bias.item()

    error = abs(weight - 2) + abs(bias - (-0.4))
    errors.append(error)

print(model.weight.item())
print(model.bias.item())

plt.plot(losses)
plt.show()

plt.plot(errors)
plt.show()

plt.plot(losses)
plt.plot(errors)
plt.show()
```


2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
20
40
60
80
100


2.0
1.5
1.0
0.5
0.0
0
20
40
60
80
100

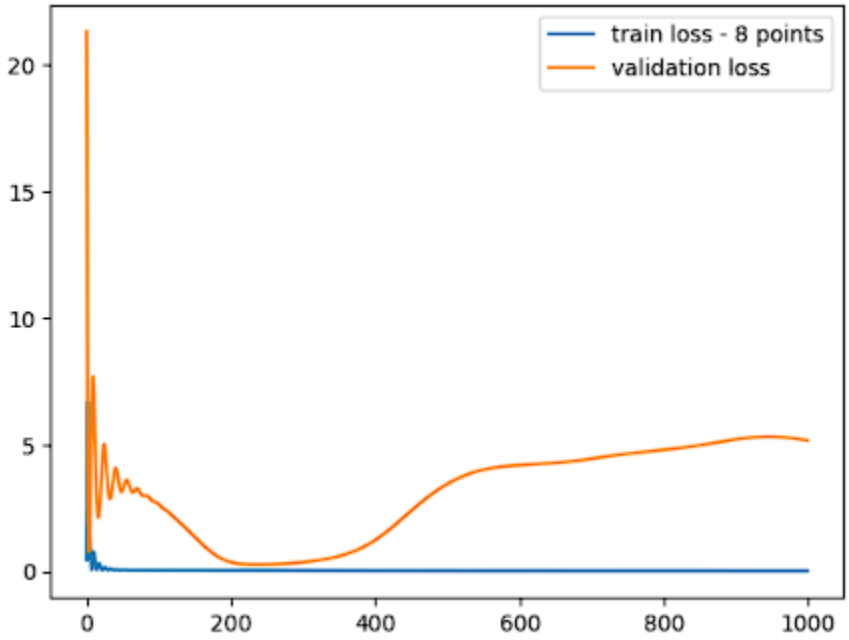

2.0
1.5
1.0
0.5
0.0
0
20
40
60
80
100

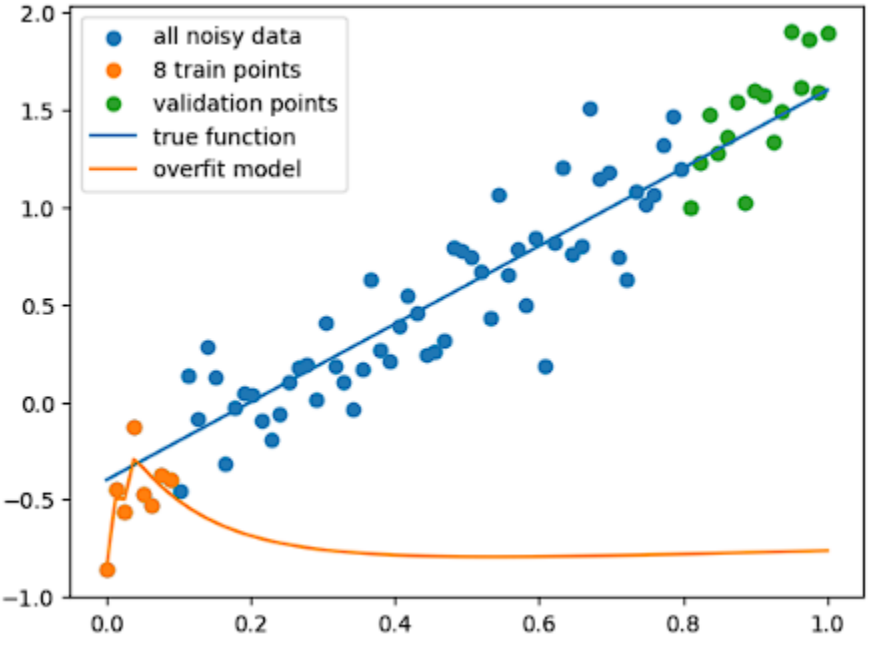
3. Split 80 points into train and validation sets. Overfit 8 noisy points with a wide MLP; explain why train loss alone misleads.

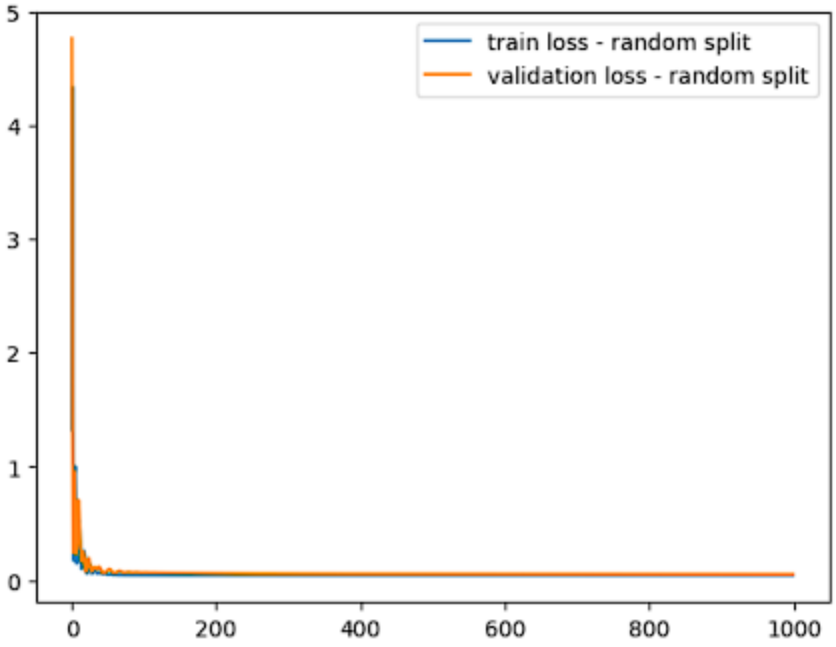
```
# practice 3 final try (**خودم**)

import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(0)

x = torch.linspace(0, 1, 80).reshape(80, 1)
y = 2 * x - 0.4

noise = 0.2 * torch.randn(80, 1)
y_noisy = y + noise

# =========================
# part 1: overfit only 8 points
# =========================

x_train = x[0:64]
y_train = y_noisy[0:64]

x_validation = x[64:80]
y_validation = y_noisy[64:80]

x_crazy_train = x_train[0:8]
y_crazy_train = y_train[0:8]

model = nn.Sequential(
    nn.Linear(1, 128),
    nn.Tanh(),
    nn.Linear(128, 128),
    nn.Tanh(),
    nn.Linear(128, 1)
)

loss_function = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

train_losses = []
validation_losses = []

for i in range(1000):
    prediction = model(x_crazy_train)
    train_loss = loss_function(prediction, y_crazy_train)

    optimizer.zero_grad()
    train_loss.backward()
    optimizer.step()

    with torch.no_grad():
```

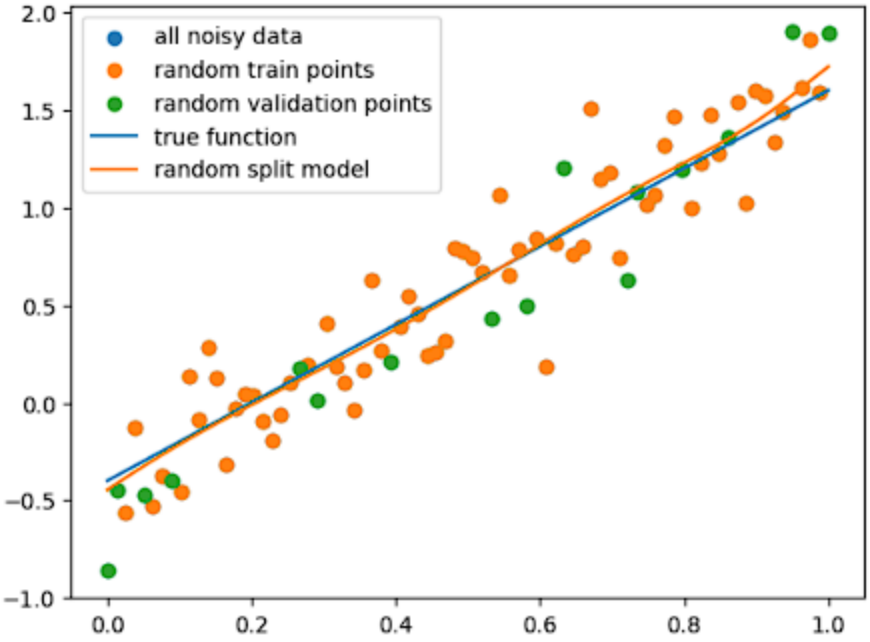
```
        validation_prediction = model(x_validation)
        validation_loss = loss_function(validation_prediction, y_validation)

    train_losses.append(train_loss.item())
    validation_losses.append(validation_loss.item())

plt.plot(train_losses, label="train loss - 8 points")
plt.plot(validation_losses, label="validation loss")
plt.legend()
plt.show()

with torch.no_grad():
    final_prediction = model(x)

plt.scatter(x, y_noisy, label="all noisy data")
plt.scatter(x_crazy_train, y_crazy_train, label="8 train points")
plt.scatter(x_validation, y_validation, label="validation points")
plt.plot(x, y, label="true function")
plt.plot(x, final_prediction, label="overfit model")
plt.legend()
plt.show()


# =========================
# part 2: random 80/20 train-validation split
# =========================

indices = torch.randperm(80)

train_indices = indices[0:64]
validation_indices = indices[64:80]

x_train_random = x[train_indices]
y_train_random = y_noisy[train_indices]

x_validation_random = x[validation_indices]
y_validation_random = y_noisy[validation_indices]

model_random = nn.Sequential(
    nn.Linear(1, 128),
    nn.Tanh(),
    nn.Linear(128, 128),
    nn.Tanh(),
    nn.Linear(128, 1)
)

optimizer_random = torch.optim.Adam(model_random.parameters(), lr=0.01)

train_losses_random = []
validation_losses_random = []

for i in range(1000):
    prediction_random = model_random(x_train_random)
    train_loss_random = loss_function(prediction_random, y_train_random)
```

```
    optimizer_random.zero_grad()
    train_loss_random.backward()
    optimizer_random.step()

    with torch.no_grad():
        validation_prediction_random = model_random(x_validation_random)
        validation_loss_random = loss_function(validation_prediction_random,
y_validation_random)

    train_losses_random.append(train_loss_random.item())
    validation_losses_random.append(validation_loss_random.item())

plt.plot(train_losses_random, label="train loss - random split")
plt.plot(validation_losses_random, label="validation loss - random split")
plt.legend()
plt.show()

with torch.no_grad():
    final_prediction_random = model_random(x)

plt.scatter(x, y_noisy, label="all noisy data")
plt.scatter(x_train_random, y_train_random, label="random train points")
plt.scatter(x_validation_random, y_validation_random, label="random validation
points")
plt.plot(x, y, label="true function")
plt.plot(x, final_prediction_random, label="random split model")
plt.legend()-
plt.show()
```


train loss - 8 points
validation loss
20
15
10
5
0
0
200
400
600
800
1000


all noisy data
8 train points
validation points
true function
overfit model
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0.0
0.2
0.4
0.6
0.8
1.0



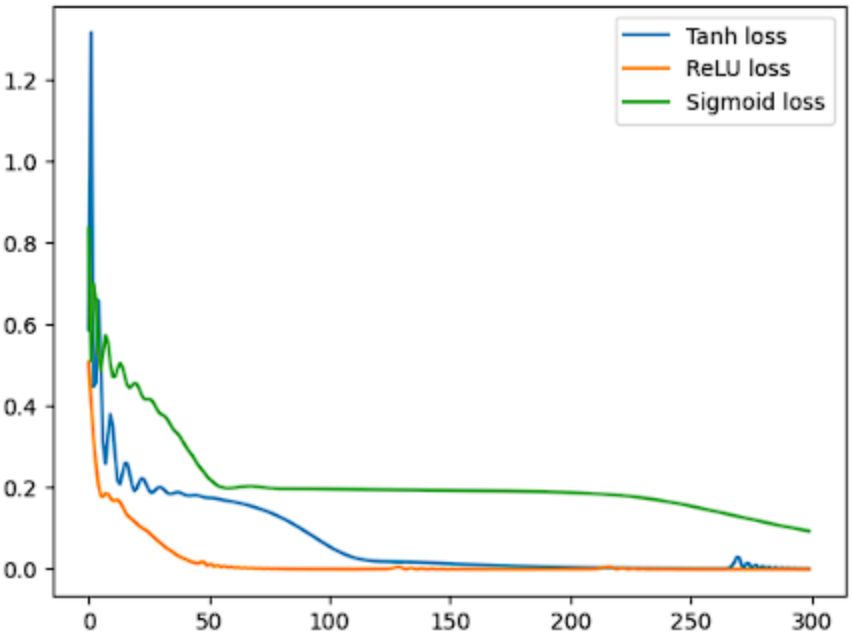


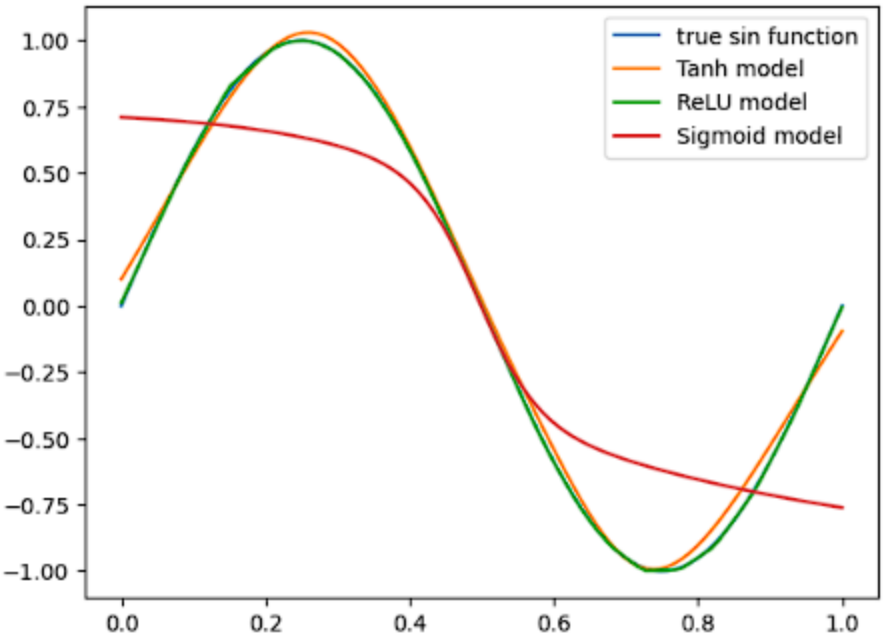
توضیح : در این سوال داده‌ها به دو بخش `train` و `validation` تقسیم می‌شوند مدل فقط با داده های `train` آموزش می‌بیند اما عملکرد واقعی آاون با `validation` بررسی می‌شود اگر `train loss` کم شود ولی `validation loss` زیاد بماند یا افزایش پیدا کند، یعنی مدل به جای یادگیری الگوی کلی، نویز های داده‌ی `train` را حفظ کرده است. به همین دلیل فقط نگاه کردن به `loss` گمراه‌کننده است توی این سوال بخاطر اینکه ما ما با استفاده از 8 داده یعنی تعداد داده ی نسبتا کم مدل رو اموزش دادیم پس مدل نتوانسته عملکرد خوبی پیدا کند و پیش بینی های درستی به ما بدهد یعنی حالت کم برازش یا اندرفیتینگ رخ میدهد.

4. Replace `tanh` by ReLU and sigmoid; compare convergence and output smoothness on $y = \sin(2\pi x)$.

```
#practice 4] final try[ (**خودم**)

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
torch.manual_seed(0)
x = torch.linspace(0, 1, 100).reshape(1 ,100)
y = torch.sin(2 * torch.pi * x)
model_tanh = nn.Sequential)
  nn.Linear,(64 ,1)
  nn.Tanh,()
  nn.Linear,(64 ,64)
  nn.Tanh,()
  nn.Linear(1 ,64)
(
model_relu = nn.Sequential)
  nn.Linear,(64 ,1)
  nn.ReLU,()
  nn.Linear,(64 ,64)
  nn.ReLU,()
  nn.Linear(1 ,64)
(
model_sigmoid = nn.Sequential)
  nn.Linear,(64 ,1)
  nn.Sigmoid,()
  nn.Linear,(64 ,64)
  nn.Sigmoid,()
  nn.Linear(1 ,64)
(
loss_function = nn.MSELoss()
optimizer_tanh = torch.optim.Adam(model_tanh.parameters(), lr=0.01)
optimizer_relu = torch.optim.Adam(model_relu.parameters(), lr=0.01)
optimizer_sigmoid = torch.optim.Adam(model_sigmoid.parameters(), lr=0.01)
losses_tanh[] =
losses_relu[] =
losses_sigmoid[] =
for i in range:(300)
  prediction_tanh = model_tanh(x)
  loss_tanh = loss_function(prediction_tanh, y)
  optimizer_tanh.zero_grad()
  loss_tanh.backward()
  optimizer_tanh.step()
  prediction_relu = model_relu(x)
  loss_relu = loss_function(prediction_relu, y)
  optimizer_relu.zero_grad()
  loss_relu.backward()
  optimizer_relu.step()
  prediction_sigmoid = model_sigmoid(x)
  loss_sigmoid = loss_function(prediction_sigmoid, y)
  optimizer_sigmoid.zero_grad()
  loss_sigmoid.backward()
  optimizer_sigmoid.step()
```

```
  losses_tanh.append(loss_tanh.item())
  losses_relu.append(loss_relu.item())
  losses_sigmoid.append(loss_sigmoid.item())

plt.plot(losses_tanh, label="Tanh loss")
plt.plot(losses_relu, label="ReLU loss")
plt.plot(losses_sigmoid, label="Sigmoid loss")
plt.legend()
plt.show()

with torch.no_grad():
  prediction_tanh = model_tanh(x)
  prediction_relu = model_relu(x)
  prediction_sigmoid = model_sigmoid(x)
plt.plot(x, y, label="true sin function")
plt.plot(x, prediction_tanh, label="Tanh model")
plt.plot(x, prediction_relu, label="ReLU model")
plt.plot(x, prediction_sigmoid, label="Sigmoid model")
plt.legend()
plt.show()
```


Tanh loss
ReLU loss
Sigmoid loss
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
50
100
150
200
250
300

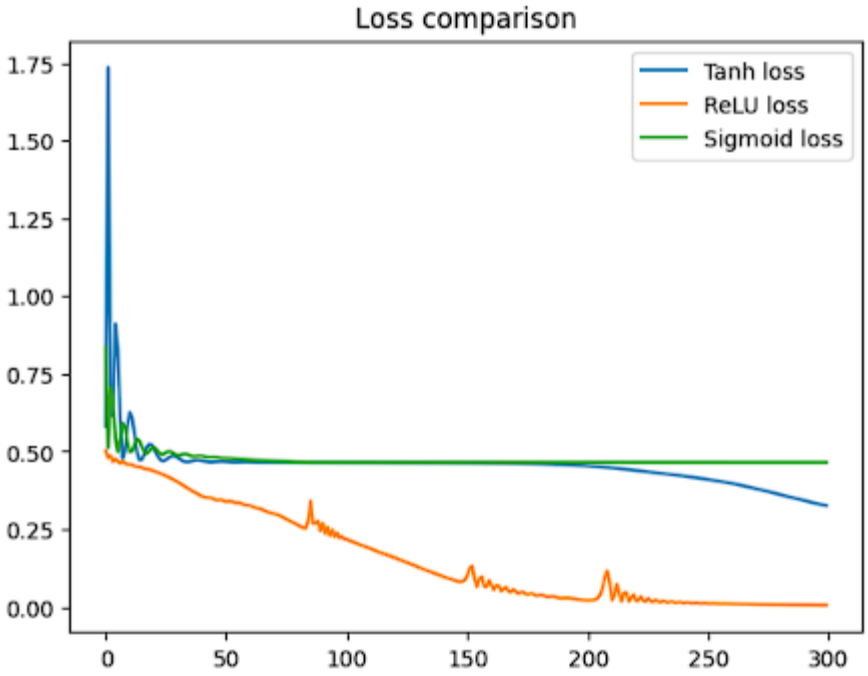

true sin function
Tanh model
ReLU model
Sigmoid model
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0.0
0.2
0.4
0.6
0.8
1.0

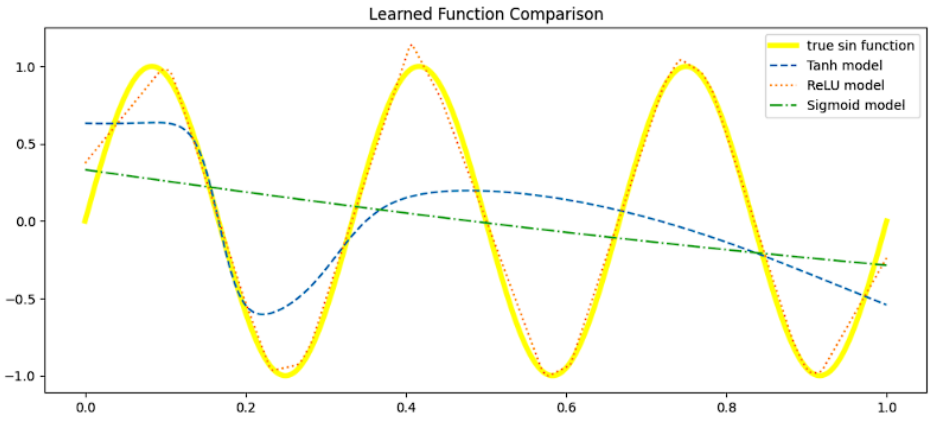
5. Finite-difference the learned function and compare its slope with the true slope.

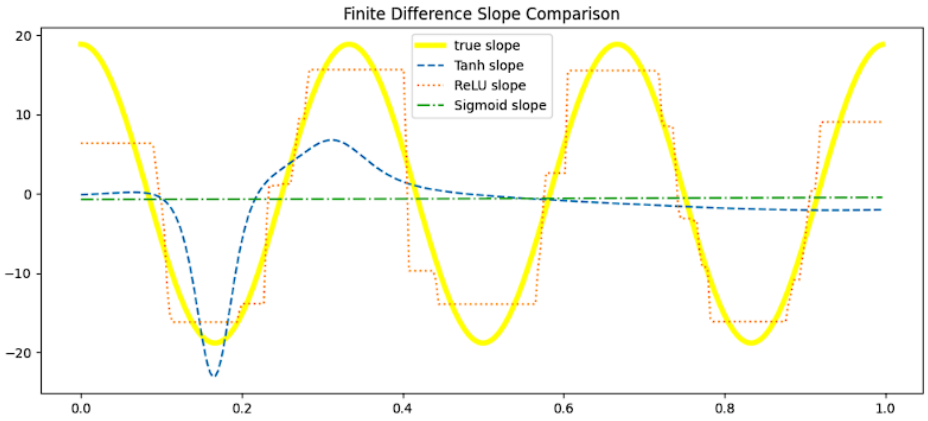
```
#practice 5 [خودم]

import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(0)

# input and true function
x = torch.linspace(0, 1, 300).reshape(300, 1)
y = torch.sin(6 * torch.pi * x)

# models
model_tanh = nn.Sequential(
    nn.Linear(1, 64),
    nn.Tanh(),
    nn.Linear(64, 64),
    nn.Tanh(),
    nn.Linear(64, 1)
)

model_relu = nn.Sequential(
    nn.Linear(1, 64),
    nn.ReLU(),
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)

model_sigmoid = nn.Sequential(
    nn.Linear(1, 64),
    nn.Sigmoid(),
    nn.Linear(64, 64),
    nn.Sigmoid(),
    nn.Linear(64, 1)
)

loss_function = nn.MSELoss()

optimizer_tanh = torch.optim.Adam(model_tanh.parameters(), lr=0.01)
optimizer_relu = torch.optim.Adam(model_relu.parameters(), lr=0.01)
optimizer_sigmoid = torch.optim.Adam(model_sigmoid.parameters(), lr=0.01)

losses_tanh = []
losses_relu = []
losses_sigmoid = []

# training
for i in range(300):
    prediction_tanh = model_tanh(x)
    loss_tanh = loss_function(prediction_tanh, y)
```

```
    optimizer_tanh.zero_grad()
    loss_tanh.backward()
    optimizer_tanh.step()

    prediction_relu = model_relu(x)
    loss_relu = loss_function(prediction_relu, y)

    optimizer_relu.zero_grad()
    loss_relu.backward()
    optimizer_relu.step()

    prediction_sigmoid = model_sigmoid(x)
    loss_sigmoid = loss_function(prediction_sigmoid, y)

    optimizer_sigmoid.zero_grad()
    loss_sigmoid.backward()
    optimizer_sigmoid.step()

    losses_tanh.append(loss_tanh.item())
    losses_relu.append(loss_relu.item())
    losses_sigmoid.append(loss_sigmoid.item())

# plot loss
plt.plot(losses_tanh, label="Tanh loss")
plt.plot(losses_relu, label="ReLU loss")
plt.plot(losses_sigmoid, label="Sigmoid loss")
plt.legend()
plt.title("Loss comparison")
plt.show()

# predictions
with torch.no_grad():
    prediction_tanh = model_tanh(x)
    prediction_relu = model_relu(x)
    prediction_sigmoid = model_sigmoid(x)

# plot learned functions
plt.figure(figsize=(12, 5))
plt.plot(x, y, label="true sin function", color="yellow", linewidth=4)
plt.plot(x, prediction_tanh, label="Tanh model", linestyle="--")
plt.plot(x, prediction_relu, label="ReLU model", linestyle=":")
plt.plot(x, prediction_sigmoid, label="Sigmoid model", linestyle="-.")
plt.legend()
plt.title("Learned Function Comparison")
plt.show()

# finite difference slopes
dx = x[1] - x[0]

slope_tanh = (prediction_tanh[1:] - prediction_tanh[:-1]) / dx
slope_relu = (prediction_relu[1:] - prediction_relu[:-1]) / dx
slope_sigmoid = (prediction_sigmoid[1:] - prediction_sigmoid[:-1]) / dx

x_slope = x[:-1]
```

```
# true slope of sin(6πx)
true_slope = 6 * torch.pi * torch.cos(6 * torch.pi * x_slope)

# plot slope comparison
plt.figure(figsize=(12, 5))
plt.plot(x_slope, true_slope, label="true slope", color="yellow", linewidth=4)
plt.plot(x_slope, slope_tanh, label="Tanh slope", linestyle="--")
plt.plot(x_slope, slope_relu, label="ReLU slope", linestyle=":")
plt.plot(x_slope, slope_sigmoid, label="Sigmoid slope", linestyle="-.")
plt.legend()
plt.title("Finite Difference Slope Comparison")
plt.show()
```


Loss comparison
Tanh loss
ReLU loss
Sigmoid loss
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
50
100
150
200
250
300


Learned Function Comparison
true sin function
Tanh model
ReLU model
Sigmoid model
1.0
0.5
0.0
−0.5
−1.0
0.0
0.2
0.4
0.6
0.8
1.0


Finite Difference Slope Comparison
true slope
Tanh slope
ReLU slope
Sigmoid slope
20
10
0
−10
−20
0.0
0.2
0.4
0.6
0.8
1.0

Debug the failure

```
model = torch.nn.Linear(1, 1)
opt = torch.optim.Adam(model.parameters(), 1e-2)
for _ in range(500):
    pred = model(x)
    loss = ((pred-y)**2).mean()
    loss.backward()
    opt.step() # BUG: what is missing?
```

Questions: Why do gradients accumulate? What pattern appears in the loss? Add `opt.zero_grad()` in the correct position and justify it.

توضیح : گرادیان ها جمع میشن چون در خود پایتورچ ما الگوریتمی نداریم که به شکل اتوماتیک بیاد و گرادیان های قبلی رو از حافظه حذف بکنه و مقدار جدید گرادیان رو بیاد و جایگزین کنه پس هر باری که ما دوباره توی این لوپ یا حلقه میایم و گرادیان رو حساب میکنیم در اصل داریم مجدد مقداری رو بهش اضافه میکنیم بدون اینکه مقدار قبلی گرادیان رو حذف کرده باشیم برای اینکار باید

opt.zero_grad()

رو قبل از محاسبه گرادیان در داخل این کد بگذاریم که توی هر لوپ ابتدا بیاد و مقدار گرادیان قبلی رو حذف بکنه و کد نهایی به این شکل میشه

```
for _ in range(500):
pred = model(x)
loss = ...

opt.zero_grad()

loss.backward()

opt.step()
```

یعنی مشکل اصلی این هست که باید قبل از loss.backward() گرادیان های قبلی پاک میشدند.

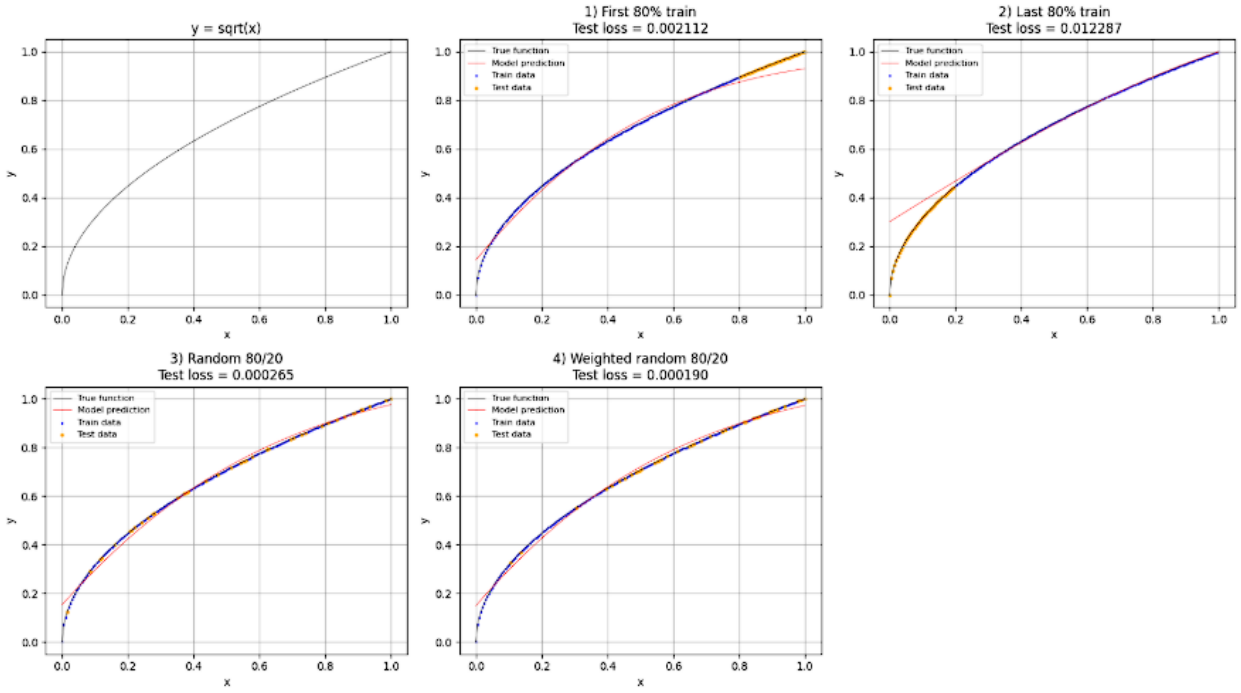
الگوی loss عمدتا در حالتی که گفته شده باعث ناپایداری میشه یعنی ممکن اون خطای ما بالا پایین بپره یا به شکل غیرعادی کم و زیاد بشه دلیلش هم اینه که گرادیان ها با هم جمع میشوند و مدل با جهت اصلاح اشتباه یا بیش از حد بزرگ اپدیت میشه .

Daily deliverable

`day1_fit.py`, one plot with train/validation curves, and a 150-word explanation of the broken loop.

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.manual_seed(0)

x = torch.linspace(0, 1, 200).reshape(-1, 1)
y = torch.sqrt(x)


def train_model(x_train, y_train):
    torch.manual_seed(0)

    model = nn.Sequential(
        nn.Linear(1, 32),
        nn.Tanh(),
        nn.Linear(32, 1)
    )

    loss_fn = nn.MSELoss()
    opt = torch.optim.Adam(model.parameters(), lr=0.01)

    for epoch in range(1000):
        prediction = model(x_train)
        loss = loss_fn(prediction, y_train)

        opt.zero_grad()
        loss.backward()
        opt.step()

    return model


train_1 = torch.arange(0, 160)
test_1 = torch.arange(160, 200)

test_2 = torch.arange(0, 40)
train_2 = torch.arange(40, 200)

torch.manual_seed(1)
random_order = torch.randperm(200)
train_3 = random_order[:160]
test_3 = random_order[160:]

torch.manual_seed(1)
weights = 1 / (x.squeeze() + 0.1)
train_4 = torch.multinomial(weights, 160, replacement=False)
```

```
all_indices = torch.arange(200)
is_train = torch.zeros(200, dtype=torch.bool)
is_train[train_4] = True
test_4 = all_indices[~is_train]

splits = [
    (train_1, test_1, "1) First 80% train"),
    (train_2, test_2, "2) Last 80% train"),
    (train_3, test_3, "3) Random 80/20"),
    (train_4, test_4, "4) Weighted random 80/20")
]


fig, axes = plt.subplots(2, 3, figsize=(16, 9))

axes[0, 0].plot(x, y, color="black", linewidth=0.5)
axes[0, 0].set_title("y = sqrt(x)")
axes[0, 0].set_xlabel("x")
axes[0, 0].set_ylabel("y")
axes[0, 0].grid(True)

plot_positions = [(0, 1), (0, 2), (1, 0), (1, 1)]

for (train_idx, test_idx, title), position in zip(splits, plot_positions):
    model = train_model(x[train_idx], y[train_idx])

    with torch.no_grad():
        full_prediction = model(x)
        test_prediction = model(x[test_idx])
        test_loss = nn.MSELoss()(test_prediction, y[test_idx]).item()

    ax = axes[position]
    ax.plot(x, y, color="black", linewidth=0.5, label="True function")
    ax.plot(x, full_prediction, color="red",linewidth=0.5, label="Model prediction")
    ax.scatter(x[train_idx], y[train_idx], s=1, color="blue", label="Train data")
    ax.scatter(x[test_idx], y[test_idx], s=5, color="orange", label="Test data")
    ax.set_title(f"{title}\nTest loss = {test_loss:.6f}")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.grid(True)
    ax.legend(fontsize=8)

axes[1, 2].axis("off")

plt.tight_layout()
plt.savefig("sqrt_five_plots.png", dpi=200)
plt.show()
```



توضیح :
یعنی حلقه‌ی آموزش یه جاییش خرابه.
اینجا مشکل اینه که قبل از `backward()` نیومدیم گرادیان‌های قبلی رو پاک کنیم. توی `PyTorch` گرادیان‌ها خودبه‌خود صفر نمی‌شن؛ هر بار که `loss.backward()` می‌زنیم، گرادیان جدید روی قبلی‌ها جمع می‌شه.
پس مدل به جای اینکه هر بار فقط با خطای همون مرحله خودش رو اصلاح کنه، با یه عالمه گرادیان جمع‌شده آپدیت می‌شه. برای همین آموزش ممکنه قاطی کنه، `loss` بالا پایین بپره، یا مدل درست یاد نگیره.

حلقه‌ی درست باید این شکلی باشه:

```
opt.zero_grad()
loss.backward()
opt.step()
```

اول خطای قبلی رو پاک می‌کنیم، بعد می‌فهمیم این بار چقدر اشتباه کردیم، بعد وزن‌های مدل رو اصلاح می‌کنیم